Short Paper: Barry: A Streaming Storage Engine for Temporal Graphs

Anonymous submission

Abstract

Temporal graphs whose edges are immutable timestamped events—audit streams, agent execution traces, interaction logs—are ingested as never-ending, near-sorted streams; queried along time and causality; and retained under time-based policies. No deployed storage layer serves this combination: entity-keyed graph stores demote the timestamp to a payload attribute, columnar and LSM engines pay a join per traversal hop, versioned temporal graph stores assume few property-rich versions per entity, and statically time-sorted layouts answer queries well while being unable to ingest at all.

Barry keys storage on time itself: each timestamp maps to a virtual address by a constant-time shift, and demand paging absorbs temporal sparsity, so no index is built or maintained. Ingestion becomes a lock-free $O(1)$ append independent of arrival order; retention becomes a page release over an address prefix; compaction is explicit and off the ingest path; queries remain correct *during* concurrent ingestion through a per-slot publication protocol; and because engine state is a pure function of the edge multiset, recovery is re-ingestion. We evaluate Barry against a columnar engine, three CSR layouts, and the most recent versioned temporal graph store, with element-wise answer verification, on fourteen datasets including a billion-edge enterprise corpus, and show that keying storage on time wins the streaming and traversal axes by orders of magnitude while conceding bounded, measured costs on the structural axis.

1 Introduction

A growing class of graphs is built from *events*: a system call touches a file, an agent invokes a tool, a host authenticates to another, a user replies to a message. Each edge carries the moment it happened, never changes afterwards, and arrives as part of a stream that does not end. The queries such graphs exist to answer are temporal and causal: *what could have influenced this state*, *what did this event go on to touch*, *what did the neighborhood look like at time t*, *how much happened in this window*. And because the value of an event fades with age, the data is governed by time-based retention.

Every mature storage engine approaches this workload with a layout built for something else. Dynamic graph stores—packed-memory-array, tree, and adjacency-list designs [2, 3, 10, 11]—key storage on source-vertex identity and treat the timestamp as payload, so any time-constrained query degrades to a structural scan with per-edge filtering, and any retention pass must find expired edges wherever vertex order scattered them. Columnar and LSM engines [9] invert the failure: ordered by time, they answer range filters well, but every hop of a traversal becomes a fresh join against the edge table. Versioned temporal graph databases [4, 7] solve a genuinely different problem—retrieving the valid *version* of an updated entity—and their machinery presumes few, property-rich versions per entity, an assumption an event stream violates by design. Finally, a statically time-sorted CSR answers temporal queries near-optimally and is not a storage engine at all: its only update path is a rebuild, during which it is blind.

Barry starts from an address-space observation: a 64-bit virtual address space is orders of magnitude larger than the time range of any recorded workload. Rather than maintaining an index from timestamp to location, Barry makes the timestamp *be* the location:

$$A(T) \;=\; A_{\text{base}} + \big((T - T_{\text{start}}) \gg S\big) \times P, \qquad (1)$$

a subtraction and a shift, with the operating system's demand paging absorbing sparsity at zero data-structure cost. The properties the workload needs then fall out of the layout rather than being engineered on top of it. Because an edge's home is computed, not searched, ingestion is a wait-free slot claim; because the computation ignores arrival order, out-of-order and late events cost nothing; because retention is a monotone time predicate, expiry is a release of an address prefix; and because nothing is ever updated in place, crash recovery collapses to replaying an append-only log through the normal insert path.

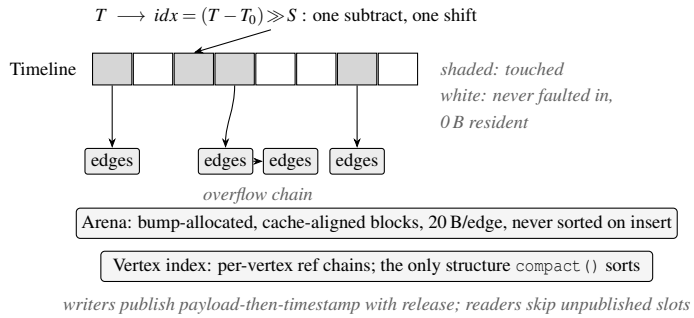


Figure 1: Barry's layout. A timestamp *computes* its timeline slot, so ingestion never searches and never sorts, and temporal sparsity is absorbed by the MMU rather than by a structure the engine maintains. Entity-keyed access lives in a separate index that compaction—never insertion—sorts.

This design has costs, and a storage paper owes their measurement as much as their mitigation. A layout that refuses to organize by entity must pay something on entity-keyed operations; a layout that never sorts on ingest must either scan unsorted neighborhoods or compact them later. We quantify both, delimit them with an explicit compaction contract—*always appendable, always correct, including mid-ingest; fast neighborhood expansion after a compaction schedulable at leisure*—and show the concessions are bounded constants while the wins are orders of magnitude (Section 4).

We contribute: (i) a storage design that keys temporal graphs on time, with formal guarantees for its addressing, space, granularity selection, concurrency, and recovery (Section 2); (ii) a concurrent-correctness protocol under which readers never observe a torn or unpublished edge during lock-free multi-writer ingestion; and (iii) an evaluation methodology that verifies element-wise answer equality across engines before reporting any latency, decomposes algorithm time into storage-read share, and measures what a static structure silently loses while rebuilding (Section 4).

2 Design

2.1 Time-keyed addressing

Equation (1) maps a timestamp to a slot in a single anonymous memory mapping (Figure 1). Two properties make this an engine rather than a trick.

Theorem 1 (Addressing)**.** *For any timestamp t, the slot holding every edge stamped t is computed in $O(1)$ worst case, with no data-dependent memory access.*

Theorem 2 (Space)**.** *With page size P and W_{occ} non-empty windows, resident timeline memory is at most $W_{occ} \cdot P$ plus lazily materialized page-table overhead; untouched windows cost zero bytes.*

Sparsity is absorbed by the MMU, not by a structure the engine maintains. The mapping is *anonymous*—no file backing—so the documented failure modes of file-backed mmap buffer management [1] (torn file pages, SIGBUS on truncation, eviction stalls) do not arise; Barry uses mmap the way an allocator reserves address space, not as a buffer pool. The shift S is chosen adaptively from stream statistics; the selected window occupancy is provably within $[\text{target}/2, 4 \cdot \text{target})$, the factor-of-two slack on each side being the honest price of power-of-two addressing. Proofs of all theorems appear in the anonymized supplemental report.

2.2 Lock-free ingestion, correct concurrent reads

An insert computes its slot (Theorem 1), claims an index in the window's current block by a single `fetch_add`, and installs overflow blocks by CAS; every CAS failure implies another insert succeeded, so ingestion is lock-free with wait-free slot claims. Nothing sorts, nothing rebalances, and arrival order is irrelevant—the property that separates Barry from every sorted-on-ingest design the moment events straggle.

Concurrent readers are protected by a publication protocol rather than locks. A writer fills an edge's payload first and stores its timestamp *last* with release semantics; since fresh arena pages are zero-filled, a zero timestamp identifies an unpublished slot, and readers acquire-load and skip it. Vertex-index refs publish through their block pointer identically. One ordering rule makes the optimized read path sound as well: an append clears the bucket's *sorted* flag *before* bumping its counter, so any reader that still observes the flag after an acquiring counter-read is provably looking at a fully compacted, fully published prefix, and may binary-search it.

Theorem 3 (Concurrent reads)**.** *Under the publication protocol, no query observes a torn, partial, or unpublished edge during concurrent multi-writer ingestion, and every query result equals that of some serially consistent state.*

2.3 The compaction contract

Appends leave a vertex's neighborhood in arrival order, so a time-windowed expansion costs $O(d)$. An explicit `compact()` pass sorts each neighborhood by timestamp once, after which expansion binary-searches in $O(\log d + k)$. Compaction is deliberately *not* on the insert path: folding it in would make every batch pay for the whole index and forfeit streaming. Queries are correct in either state—compaction changes cost, never answers. The resulting contract is the paper's central trade: a statically sorted structure gets its speed by making sorting *mandatory* and paying with a blind rebuild window (Section 4.3); Barry makes sorting *optional* and pays a bounded constant until it runs.

2.4 Retention and durability

Retention is a monotone time predicate, so under Equation (1) it is an address-prefix operation: `evict_before(t)` walks the timeline prefix, merges block extents, and returns whole pages with `madvise`; readers of other windows are undisturbed. For durability, Barry exploits a property databases rarely enjoy: engine state is a pure function of the edge multiset. The write-ahead log is therefore just framed, checksummed batches of raw edges, an asynchronous writer overlaps it with insertion, and recovery is replaying the log through the normal insert path—no undo, no redo, no idempotence bookkeeping; a torn tail frame is detected and discarded.

3 Implementation

Barry is ∼6,300 lines of dependency-free C++17: the engine, nine graph algorithms including bidirectional time-respecting traversal, converters for two audit-log formats, and seven correctness suites (among them a multi-writer/multi-reader stress test of Theorem 3 and an end-to-end synthetic-provenance fixture). Interior windows of a range scan are copied without any timestamp comparison—only boundary windows can contain out-of-range edges—and boundary scans are AVX2-vectorized over the packed edges. Source, converters, proofs, and every experiment driver will be released.

4 Evaluation

Setup. 20-core Xeon, 188 GB RAM, Ubuntu 20.04. Fourteen datasets: LANL enterprise authentication (1.05B events, 17,666 hosts, 58 days) [5], DARPA TC whole-system provenance (CADETS 22.5M and THEIA 97.3M information-flow edges over 2.9M/6.9M entities), and 12 SNAP temporal graphs (60K–646M edges). Baselines: DuckDB v1.5.5 through its C API in the same process; a time-sorted CSR (`tcsr`, the strongest static competitor); an arrival-order entity CSR (`ucsr`, what an ingesting entity-keyed store holds between rebuilds); the topology-only CSR; and TVA [7], the most recent PVLDB temporal graph store, via its released artifact. Every cross-engine latency in this section is preceded by element-wise answer verification. Latency distributions on this workload are heavy-tailed; we report medians with tails or size-binned results, never means.

4.1 Ingestion

Barry ingests at 11.9–15.5M events/s single-threaded across wiki-talk, CADETS, and THEIA (Table 1), and sustains 6.8M events/s on the billion-edge LANL corpus under 12 concurrent writers *while* 4 readers issue validated queries (48,636 queries, 780M edges checked, zero protocol violations, final count exact). With the write-ahead log enabled, durable ingestion reaches 4.2M events/s under 64 MB group commit against 7.4M volatile—bounded by the disk's measured 156 MB/s sequential bandwidth, not by the engine—and 1.2M with an fsync per batch. Driving TVA's artifact with the same single-threaded event stream did not complete wiki-talk within a 90-minute cutoff at 41.6 GB RSS (Figure 2): under event streams, every repeated (src, dst) contact extends one edge's version chain, the structure its hopscotch layout must keep within bounded offsets, and its design assumption—few, property-rich versions per entity—inverts. A statically sorted CSR is absent from this comparison for the simplest reason: it has no ingest path at all.

Table 1: Cross-engine operations, identical data and thread count. Snapshot = full-graph as-of scan. TVA row is a bound (DNF at the 90-minute cutoff); a sorted CSR has no ingest path to measure.

Engine	Dataset	Ingest	Pt. scan	Snapshot	RSS
Barry	wiki-talk	0.60 s	0.2 µs	0.65 s	0.7 GB
Barry	CADETS	1.45 s	0.1 µs	218 s	1.8 GB
Barry	THEIA	8.16 s	0.4 µs	45.8 s	7.7 GB
TVA	wiki-talk	>5,415 s	—	—	>41.6 GB

4.2 Time-respecting traversal

On multi-hop time-respecting traversal run to fixpoint, Barry leads DuckDB by 150–700× across all answer sizes—at $|R| > 1{,}000$ the median is 12 ms against 2.43 s—because each hop costs a relational engine a join against the edge table (Figure 3a). The time-sorted CSR remains 2.7× faster than Barry at high degree: that is the measured price of refusing to sort on ingest, and Section 4.3 prices what the CSR pays for the privilege. Against the *ingesting* entity-keyed layout the ordering flips with degree (Figure 3b): at mean degree 20 `ucsr` wins on contiguity, at 337 Barry reaches parity, and at LANL's mean degree of 59,517 Barry leads by 104× (4.2 ms vs. 435 ms)—the crossover lands where event-stream workloads actually live. On time-*range* filtering, DuckDB's home ground, Barry's vectorized scan wins the median 10.6× (77 µs vs. 819 µs, five seeds) with p95 at parity (981 vs. 1,105 µs, overlapping ranges). Decomposing classic graph algorithms (BFS, PageRank, label propagation) into storage-read share under identical, checksum-verified executions bounds the structural concession: Barry delivers adjacency at 0.9–2.9× the topology-only CSR optimum (median 1.7×), and the ratio *improves* with scale—1.4× on the billion-edge corpus, where long bucket chains amortise—while Barry is the only layout of the four that ingests at all (Figure 4). Barry's weakest operation is the full-graph as-of snapshot (enumerate-then-reduce; 218 s on

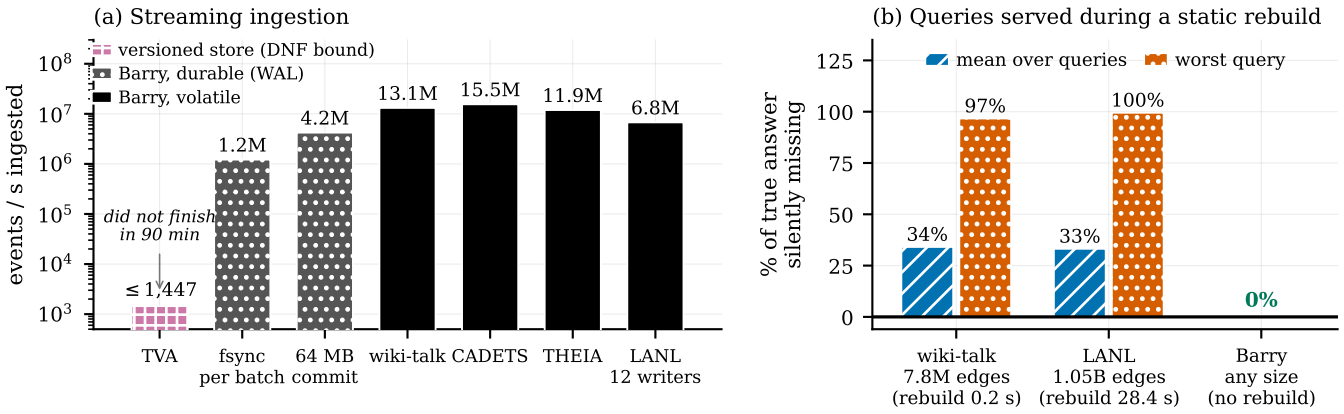


Figure 2: The dynamic axis. **(a)** Single-thread ingestion unless noted; log scale. The versioned store did not finish wiki-talk's 7.8M events within a 90-minute cutoff, so its bar is an upper bound; a statically sorted CSR has no ingest path to place on this axis at all. Barry's durable modes are bounded by the disk's 156 MB/s sequential bandwidth, not by the engine. **(b)** Queries answered from a static structure's pre-rebuild snapshot are not stale but silently incomplete; Barry never has a rebuild window, at any corpus size.

CADETS, whose hottest vertices carry millions of events): precisely the operation TVA's version-skipping is built to serve, and we report it as its win.

4.3 Rebuild versus stream

The time-sorted CSR's only update path is a full rebuild (28.4 s on LANL). Queries about the live window served from the pre-rebuild snapshot are not stale but silently *wrong*: on LANL, 68 of 100 queries returned incomplete answers, losing 33.2% of the true result on average and 99.5% in the worst case; an independent wiki-talk measurement gives 34.1%/96.7% (Figure 2b). Barry ingests the same arrivals incrementally at 3.3M events/s and has no blind window. Barry's own optional compaction costs more than the CSR rebuild (76 s vs. 28 s on LANL; an arena-order timestamp resolve would close this and is future work)—but it sits on the performance path, not the correctness path, which is the entire point of the contract.

4.4 Retention, scale, and a real workload

`evict_before` releases 90% of a 16.5M-edge history in 18 ms and returns 331 MB of resident memory, 5–29× faster than a columnar `DELETE`; both reclaim memory fully, so the win is speed alone. LANL loads in 255 s at 96 GB RSS with the reverse index; the 646M-edge reddit corpus runs in 39.6 GB. As an end-to-end grounding on real ground truth: tracing all 749 labelled red-team alerts of the LANL corpus to causal fixpoint takes 46.5 s total (median 24.6 μs, p95 353 ms), and on DARPA CADETS a forward trace from the documented exploit source reconstructs the full published attack chain—implant drop, privilege escalation, C2, injection target—in causal order within an impact set of 0.0093% of the graph, at 133 μs.

4.5 Negative results, kept

For the record: caching timestamps in vertex-index refs (+50% index memory for 15% traversal) was built, measured, and reverted; binary-search expansion buys 2.5× at degree 337 and nothing below degree 30; and the retention comparison's original hypothesis—that the columnar engine would fail to reclaim memory—was false.

5 Related Work

Versioned temporal graph stores. TVA [7] and AeonG [4] organize versions of *updated entities* for snapshot retrieval; delta encoding and version-chain locality, their central difficulties, do not arise in an append-only event stream, and time-respecting reachability is not expressible over their snapshot query surface. The two lines are complementary, and measurably so in both directions (Sections 4.1 and 4.2): neither subsumes the other; they store different things. **Structural dynamic graph stores** [2, 3, 10, 11] optimize the entity axis and treat time as payload; the storage-read decomposition bounds what Barry concedes on their axis, and none answers time-respecting queries natively. **Columnar/LSM engines** [9] own predictable range-filter tails and lose traversal by orders of magnitude. **mmap in databases.** The case against mmap [1] targets file-backed buffer management; Barry's anonymous reservation sidesteps each cited failure mode. **Provenance systems** [6, 8, 12] motivate the workload's rates and retention stakes; serving them is an application of this engine, developed separately.

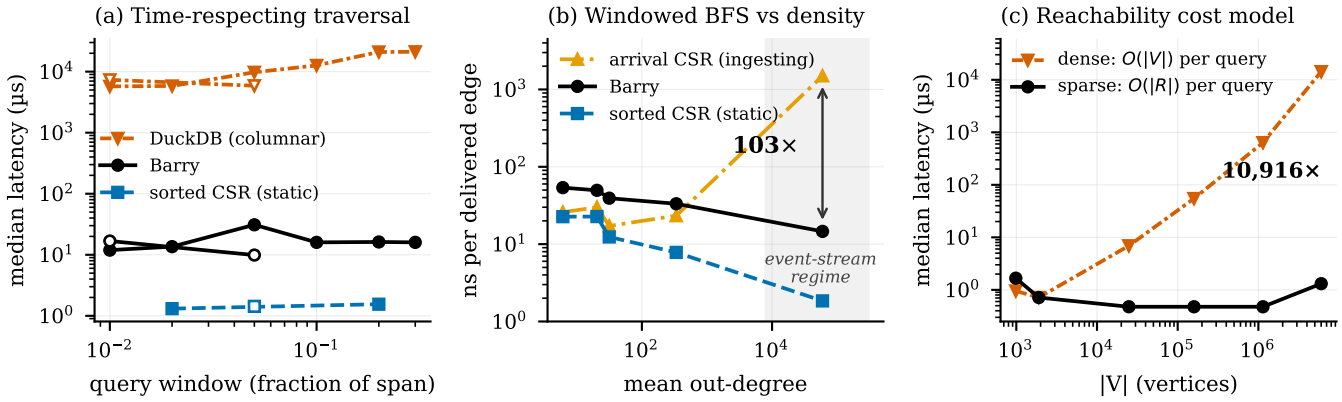


Figure 3: The time axis, three ways. **(a)** Time-respecting traversal to fixpoint across query windows on two corpora (filled markers: sx-mathoverflow; open: wiki-talk); answers verified element-wise identical across engines before timing. **(b)** Windowed BFS, normalised to cost *per delivered edge*—the only size-independent comparison across corpora spanning 60K to 1.05B edges. An arrival-order adjacency must scan an entire neighborhood to find the in-window edges, so its per-edge cost explodes with density (26 → 1500 ns); Barry's compacted buckets amortise a binary search, so its cost *improves* (54 → 15 ns). The crossover lands inside the regime event streams occupy. **(c)** Sparse temporal reachability makes query cost track the answer rather than the graph.

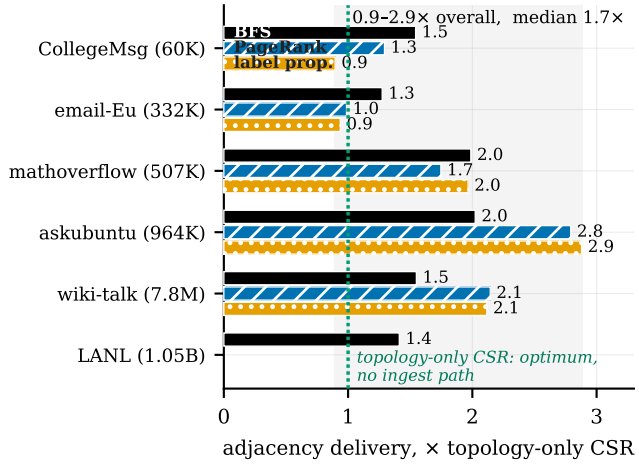


Figure 4: The concession, priced. Adjacency delivery for structure-only algorithms against the topology-only CSR optimum, under identical checksum-verified executions. The cost is a small constant that *improves* with scale, and it buys an ingest path the optimum does not have.

6 Conclusion

Keying graph storage on time collapses the machinery an event stream otherwise requires: the index becomes a shift, ingestion an order-indifferent lock-free append, retention a page release, recovery a replay, and temporal traversal an address computation—correct even mid-ingest. The concessions are bounded constants, measured at the storage layer where they belong; the wins are the axes an event stream cannot live without.

References

[1] Andrew Crotty, Viktor Leis, and Andrew Pavlo. Are you sure you want to use MMAP in your database management system? In *Proc. CIDR*, 2022.

[2] Laxman Dhulipala, Guy E. Blelloch, and Julian Shun. Low-latency graph streaming using compressed purely-functional trees. In *PLDI*, 2019.

[3] Per Fuchs, Domagoj Margan, and Jana Giceva. Sortledton: A universal graph data structure. *ACM SIGMOD Record*, 2023.

[4] Jiamin Hou, Zhanhao Zhao, Zhouyu Wang, Wei Lu, Guodong Jin, Dong Wen, and Xiaoyong Du. Aeong: An efficient built-in temporal support in graph databases. *PVLDB*, 17(6), 2024.

[5] Alexander D. Kent. Comprehensive, Multi-Source Cyber-Security Events. Los Alamos National Laboratory, 2015.

[6] Samuel T. King and Peter M. Chen. Backtracking intrusions. In *Proc. SOSP*, 2003.

[7] Wenhao Li, Zhanhao Zhao, Jinhao Dong, Jiamin Hou, Wei Lu, Yunhai Wang, and Xiaoyong Du. TVA: A version-aware temporal graph storage system for real-time analytics. In *PVLDB*, volume 19, 2026.

[8] Thomas Pasquier, Xueyuan Han, Mark Goldstein, Thomas Moyer, David Eyers, Margo Seltzer, and Jean Bacon. Practical whole-system provenance capture. In *Proc. SoCC*, 2017.

[9] Mark Raasveldt and Hannes Mühleisen. DuckDB: An embeddable analytical database. In *Proc. SIGMOD*, 2019.

[10] Yue Shen, Jie Zhang, Huawei Cao, Yuan Zhang, and Xuejun An. Dcsr: A fast data structure with leaf-oriented locks for streaming graph processing. In *EDBT*, 2026.

[11] Brian Wheatman and Helen Xu. A parallel packed memory array to store dynamic graphs. In *ALENEX*, 2021.

[12] Bo Zhang et al. TAPAS: An efficient online APT detection with task-guided process provenance graph segmentation and analysis. In *Proc. USENIX Security*, 2025.